CI/CD

What is CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery (or Continuous Deployment).

What is CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery (or Continuous Deployment).
- It is a set of practices and tools that help developers deliver code changes more frequently and reliably.

What is CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery (or Continuous Deployment).

- It is a set of practices and tools that help developers deliver code changes more frequently and reliably.

  - CI (Continuous Integration): Developers frequently integrate their code into a shared repository (like GitHub/GitLab/Bitbucket).
    - Each integration is verified automatically using builds and tests.

What is CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery (or Continuous Deployment).

- It is a set of practices and tools that help developers deliver code changes more frequently and reliably.

  - CI (Continuous Integration): Developers frequently integrate their code into a shared repository (like GitHub/GitLab/Bitbucket).
    - Each integration is verified automatically using builds and tests.

  - CD (Continuous Delivery/Deployment): Once code is integrated and tested, it is automatically delivered (or deployed) to production or staging environments.
    - Continuous Delivery: Code is ready to be deployed anytime (manual approval step).
    - Continuous Deployment: Code is deployed automatically after tests pass (no manual step).

Why is CI/CD needed?

- Traditional software development had these problems:
  - Developers worked in isolation → integration issues later ("integration hell").

Why is CI/CD needed?

- Traditional software development had these problems:
  - Developers worked in isolation → integration issues later ("integration hell").
  - Manual testing and deployment took a lot of time.

Why is CI/CD needed?

- Traditional software development had these problems:
  - Developers worked in isolation → integration issues later ("integration hell").
  - Manual testing and deployment took a lot of time.
  - Bugs reached production because testing wasn't automated.

Why is CI/CD needed?

- Traditional software development had these problems:
  - Developers worked in isolation → integration issues later ("integration hell").
  - Manual testing and deployment took a lot of time.
  - Bugs reached production because testing wasn't automated.
  - Deployments were risky and done rarely → large releases caused failures.

Why is CI/CD needed?

- Traditional software development had these problems:
  - Developers worked in isolation → integration issues later ("integration hell").
  - Manual testing and deployment took a lot of time.
  - Bugs reached production because testing wasn't automated.
  - Deployments were risky and done rarely → large releases caused failures.

- CI/CD solves this by making integration, testing, and deployment automated, faster, and safer.

Advantages of CI/CD

1. Faster feedback loop: Bugs are caught early since every commit is tested.
2. Reduced integration issues: Frequent merges → less conflict between developers.
3. Automated testing ensures quality: Every change is verified.
4. Faster release cycles: Teams can release features more frequently.
5. Smaller, safer changes: Frequent small releases reduce risk compared to big-bang releases.
6. Developer productivity: Less time wasted on manual deployments.

What if we don't use CI/CD?

- Developers integrate late → conflicts and merge failures.
- Bugs remain hidden until very late.
- Manual testing takes too long → slower releases.
- High risk during production deployment.
- Customers wait longer for new features.
- In worst cases → "works on my machine" problem.

CI (Continuous Integration) with Example

- Developers push code to GitHub.

- A CI pipeline (e.g., GitHub Actions, Jenkins, GitLab CI) runs automatically:
  1. **Fetch the new code.**
  2. **Install dependencies and compile/interpret to generate machine code (Python generates bytecode).**
  3. **Run unit tests and linting (Linting first).**
  4. **Generate a build (e.g., Docker image, JAR file, etc.).**

- If tests pass → code is marked good.

- If tests fail → developer is notified immediately.

- Example:
  - We're building a Flask app.
  - Each time we git push, GitHub Actions runs:
    - pytest for automated testing.
    - flake8 for code style.
    - If everything passes, it builds a Docker image.

CD (Continuous Delivery/Deployment) with Example

- Once CI succeeds, CD takes over:

1. Continuous Delivery (manual step):
   - The tested build is automatically sent to a staging environment.
   - A manager/dev can press a button to deploy it to production.
   - Example: Our Flask app is deployed to a staging server, and after approval, it goes live.

CD (Continuous Delivery/Deployment) with Example

- Once CI succeeds, CD takes over:

1. Continuous Delivery (manual step):
   - The tested build is automatically sent to a staging environment.
   - A manager/dev can press a button to deploy it to production.
   - Example: Our Flask app is deployed to a staging server, and after approval, it goes live.

2. Continuous Deployment (fully automatic):
   - The code goes directly to production if tests pass.
   - No human approval needed.
   - Example: We push code to GitHub → Tests pass → Docker container is deployed automatically to AWS ECS/Kubernetes → Live site updated.

Categories of CI/CD Tools

Category	Purpose	Examples	Language-specific?
CI/CD Platforms	Automate build, test, deploy pipelines	Jenkins, GitHub Actions, GitLab CI/CD, Bitbucket Pipelines, CircleCI, Azure DevOps, Travis CI, AWS CodePipeline, Google Cloud Build	No (language-agnostic)
Build Tools	Compile, package, manage dependencies	Maven, Gradle (Java), setuptools/Poetry (Python), npm/Yarn/Webpack (JavaScript), CMake/Make (C/C++), MSBuild (C#/.NET)	Yes
Testing & Quality Tools	Run tests, check code quality, security scanning	JUnit/TestNG (Java), PyTest/unittest (Python), Jest/Mocha (JS), ESLint, Flake8, SonarQube, Snyk	Mostly language-specific
Containerization & Deployment Tools	Package apps into containers and deploy	Docker, Kubernetes, Helm, Terraform, Ansible	No (infrastructure-agnostic)
Monitoring & Observability Tools	Monitor deployments, logs, performance	Prometheus, Grafana, ELK Stack, Datadog	No

Example: Python Flask App with Jenkins

Using Jenkins (on-prem or server-based CI/CD)

1. Developer pushes code to GitHub.
2. Jenkins detects the push via **webhook**.
3. Jenkins pipeline stages:
   - **Build:** pip install -r requirements.txt (install dependencies).
   - **Test:** pytest (run tests).
   - **Package:** docker build -t flask-app . (create Docker image).
   - **Deploy:** Push Docker image to registry (DockerHub/ECR), then deploy to AWS EC2 or Kubernetes.

Example: Python Flask App with GitHub Actions

Using GitHub Actions (cloud CI/CD inside GitHub)

1. Developer pushes code to GitHub.
2. GitHub Actions workflow triggers (defined in .github/workflows/ci-cd.yml).
3. Workflow steps:
   - **Build:** Install dependencies with pip.
   - **Test:** Run tests with pytest.
   - **Package:** Build Docker image.
   - **Deploy:** Deploy image to AWS ECS/EKS or Heroku.

Difference

- Jenkins = you manage the CI/CD server (flexible but more setup).
- GitHub Actions = fully cloud-managed (easier, integrated with GitHub).

Jenkins

- Install Java (JDK 17/21 recommended): We can download Windows x64 Installer from (https://download.oracle.com/java/17/archive/jdk-17.0.12_windows-x64_bin.msi) or (https://download.oracle.com/java/21/archive/jdk-21.0.7_windows-x64_bin.msi).

Jenkins

- Install Java (JDK 17/21 recommended): We can download Windows x64 Installer from (https://download.oracle.com/java/17/archive/jdk-17.0.12_windows-x64_bin.msi) or (https://download.oracle.com/java/21/archive/jdk-21.0.7_windows-x64_bin.msi).

- Install Jenkins (LTS recommended): Download Jenkins .msi (Windows Installer) from: https://www.jenkins.io/download/ (Select an option for Windows). Install through msi file. While installing, in Logon Type, select 'Run Service as LocalSystem'. Then, on the next screen, click "Test Port" and then click "Next." On the next screen, we need to select path to JDK or JRE. If we have a Java folder in both the Program Files and Program Files (x86) folders, it means we have both 64-bit Java and 32-bit Java installed in these respective paths. Jenkins is a server application. It requires 64-bit Java to run reliably, especially for plugins and memory usage. Therefore, select the path to the Program Files/Java/jdk-17. Go ahead with the default selection on the next screen, and click Install.

Jenkins

- Unlock Jenkins: Open Jenkins in the browser: http://localhost:8080. Initial username is admin. Copy initial admin password from: C:\Program Files\Jenkins\secrets\initialAdminPassword. Paste it. Click on Install suggested plugins. Create an Admin User and log in.

Jenkins

- The project structure looks like the following.

```
demo_project/
    ├── dbapp/
    │   └── app.py
    │   └── Dockerfile
    │   └── .dockerignore
    ├── webapp/
    │   ├── app.py
        ├── test_sample.py
    │   ├── Dockerfile
    │   └── .dockerignore
    │   └── model.pkl
    │   └── templates/
    │       ├── index.html
    │       └── display_records.html
    ├── docker-compose.yml
    └── Jenkinsfile
```

Jenkins

- Make the root of the project a git repo.
- Open the project in the Cursor. Change directory to dbapp. Do
  - uv init --python 3.13.5 (this will also create a .git folder, remove it as we want the project root to be a git repo)
  - uv venv .venv
  - .\.venv\Scripts\activate
  - uv add flask flask_sqlalchemy
  - uv pip freeze > requirements.txt.
  - Create Dockerfile and .dockerignore files (Ignore: .venv/, .git/, .gitignore, .python-version, main.py, pyproject.toml, README.md, uv.lock).
  - deactivate
  - Change the directory to webapp and do the same thing. Extra packages are required in webapp: uv add flask numpy scikit-learn requests pytest.

Jenkins

docker-compose.yml:

```
version: "3.8"
services:
  dbapp:
    build: ./dbapp
    container_name: cntdbapp
    ports:
      - "8001:8001"

  webapp:
    build: ./webapp
    container_name: cntwebapp
    ports:
      - "8000:8000"
    depends_on:
      - dbapp
```

Jenkins – More on Docker-Compose file

- By default, Docker Compose creates a single network per Compose project, which is named <project_name>_default. We can also explicitly set via the -p or --project-name flag when we run docker-compose.

- Each service is added to it with its service name as the hostname. So containers can resolve and communicate with each other using service names — that's the correct and reliable method. Use service names like http://dbapp:8001 in the code. Avoid using container_name in hostnames or code. We can remove the container_name: lines from our Compose file for cleaner and more flexible configurations.

More on Jenkins

- Node: A Node in Jenkins refers to any machine that is part of the Jenkins distributed build system. This includes the Jenkins controller (master) itself and any connected worker machines. Nodes provide the computational resources (CPU, memory, disk space) for executing Jenkins jobs.

More on Jenkins

- Node: A Node in Jenkins refers to any machine that is part of the Jenkins distributed build system. This includes the Jenkins controller (master) itself and any connected worker machines. Nodes provide the computational resources (CPU, memory, disk space) for executing Jenkins jobs.

- Agent: An Agent, also known as a worker node or slave, is a specific type of Node where Jenkins executes build tasks or pipeline steps. Agents are responsible for performing the actual work of a Jenkins job, such as compiling code, running tests, or deploying applications.

More on Jenkins

- Node: A Node in Jenkins refers to any machine that is part of the Jenkins distributed build system. This includes the Jenkins controller (master) itself and any connected worker machines. Nodes provide the computational resources (CPU, memory, disk space) for executing Jenkins jobs.

- Agent: An Agent, also known as a worker node or slave, is a specific type of Node where Jenkins executes build tasks or pipeline steps. Agents are responsible for performing the actual work of a Jenkins job, such as compiling code, running tests, or deploying applications.

- "Node" is a broader term encompassing all machines in the Jenkins environment, while "Agent" specifically refers to the worker machines that execute tasks. The Jenkins controller is a node, but it's typically not referred to as an agent for running builds (though it can be configured to do so, which is generally discouraged).

More on Jenkins

- Agents are specifically configured to run Jenkins tasks and connect to the Jenkins controller. Nodes simply represent a machine within the Jenkins setup. In essence, all Agents are Nodes, but not all Nodes are Agents (e.g., the Jenkins controller is a node but is typically not used primarily as an agent for running builds). The term "agent" emphasizes the role of a machine as a worker for executing Jenkins jobs.

More on Jenkins

- Agents are specifically configured to run Jenkins tasks and connect to the Jenkins controller. Nodes simply represent a machine within the Jenkins setup. In essence, all Agents are Nodes, but not all Nodes are Agents (e.g., the Jenkins controller is a node but is typically not used primarily as an agent for running builds). The term "agent" emphasizes the role of a machine as a worker for executing Jenkins jobs.

- In Declarative Pipelines, the agent directive is used to specify where the entire pipeline or a specific stage should run. For example, "agent any" means the pipeline can run on any available agent, while "agent { label 'my-label' }" specifies that it will run on an agent with the specific label "my-label".

More on Jenkins

- Labels (in Jenkins UI):
  - Can be found at Manage Jenkins → Nodes → Built-In Node → Configure → set **Labels** to local.

More on Jenkins

- Labels (in Jenkins UI):
  - Labels (or tags) are used to group multiple agents into one logical group. For example, if we have multiple Windows agents and a job that must run on Windows, we can configure all our Windows agents to have the label 'Windows' and then tie that job to this label. This would ensure that our job runs on one of your Windows agents, but not on any agents without this label.

More on Jenkins

- Labels (in Jenkins UI):
  - Labels (or tags) are used to group multiple agents into one logical group. For example, if we have multiple Windows agents and a job that must run on Windows, we can configure all our Windows agents to have the label 'Windows' and then tie that job to this label. This would ensure that our job runs on one of your Windows agents, but not on any agents without this label.

  - Labels do not necessarily have to represent the operating system on the agent; we can also use labels to note the CPU architecture or that a certain tool is installed on the agent.

More on Jenkins

- Labels (in Jenkins UI):
  - Labels (or tags) are used to group multiple agents into one logical group. For example, if we have multiple Windows agents and a job that must run on Windows, we can configure all our Windows agents to have the label 'Windows' and then tie that job to this label. This would ensure that our job runs on one of your Windows agents, but not on any agents without this label.

  - Labels do not necessarily have to represent the operating system on the agent; we can also use labels to note the CPU architecture or that a certain tool is installed on the agent.

  - A space must separate multiple labels. For example, Windows Docker would assign two labels to the agent: windows and docker.

More on Jenkins

- Labels (in Jenkins UI):
  - Labels (or tags) are used to group multiple agents into one logical group. For example, if we have multiple Windows agents and a job that must run on Windows, we can configure all our Windows agents to have the label 'Windows' and then tie that job to this label. This would ensure that our job runs on one of your Windows agents, but not on any agents without this label.

  - Labels do not necessarily have to represent the operating system on the agent; we can also use labels to note the CPU architecture or that a certain tool is installed on the agent.

  - A space must separate multiple labels. For example, Windows Docker would assign two labels to the agent: windows and docker.

  - Labels may contain any non-space characters, but we should avoid special characters such as any of these: !&|<>(), as other Jenkins features allow for defining label expressions, where these characters may be used.

More on Jenkins

- Usage (in Jenkins UI): Controls how Jenkins schedules builds on this node.
  - Can be found at Manage Jenkins → Nodes → Built-In Node → Configure → Usage.

More on Jenkins

- Usage (in Jenkins UI): Controls how Jenkins schedules builds on this node.
  - Use this node as much as possible:
    - This is the default setting. In this mode, Jenkins uses this node freely. Whenever there is a build that can be done by using this node, Jenkins will use it.

More on Jenkins

- Usage (in Jenkins UI): Controls how Jenkins schedules builds on this node.
  - Use this node as much as possible:
    - This is the default setting. In this mode, Jenkins uses this node freely. Whenever there is a build that can be done by using this node, Jenkins will use it.
  - Only build jobs with label expressions matching this node:
    - In this mode, Jenkins will only build a project on this node when that project is restricted to certain nodes using a label expression, and that expression matches this node's name and/or labels.
    - This allows a node to be reserved for certain kinds of jobs. For example, if we have jobs that run performance tests, we may want them to only run on a specially configured machine, while preventing all other jobs from using that machine.
    - To do so, we would restrict where the test jobs may run by giving them a label expression matching that machine.
    - Furthermore, if we set the # of executors value to 1, we can ensure that only one performance test will execute at any given time on that machine; no other builds will interfere.

A Sample Jenkins file

```
pipeline {
   agent { //agent any does not allow specifying the custom workspace
         node
         {
            label 'local'
            customWorkspace 'D:\\Departmental Data\\Subjects Data\\July-Dec 2025\\MLOPs\\CI-CD\\Demo1_Jenkins_as_Container' /*as our project is not at the standard path*/
         }
       }

   stages {
      /*
      stage('Checkout Code') {
         steps {
            git branch: 'main', url: 'https://github.com/your-username/iris_project.git'
         }
      }*/

      stage('Show Workspace') {
         steps {
            bat 'cd'  #prints CWD
         }
      }
```

A Sample Jenkins file

```
    stage('Build Images') {
        steps {
            bat 'docker-compose build'
        }
    }

    /*stage('Build Images') {
        steps {
            sh 'docker-compose build'
        }
    }*/

    stage('Run Tests') {
        steps {
            bat 'docker run --rm demo1_jenkins_as_container_webapp pytest || echo "No tests found"'
        }
    }

    stage('Deploy Containers') {
        steps {
            bat 'docker-compose up -d'
        }
    }
  }
}
```

A Sample Jenkins file

- For the above to work, Manage Jenkins → Nodes → Built-In Node → Configure → set Labels to local. This assumes that our project is available locally and is not a Git repository. We set the pipeline definition as "Pipeline Script"

A Sample Jenkins file

If the project is available locally and it is a git repo, then we need to add the following stage under stages as the first stage:

```
stage('Checkout Code') {
   steps {
      git branch: 'main', url: 'file:///D:/Your/Full/Path/demo_project'
   }
}
```

Git URL for Jenkins should be: file:///D:/Departmental%20Data/Subjects%20Data/July-Dec%202025/MLOPs/CI-CD/demo_project_name

The following

```
agent {
        node
        {
           label 'local'
           customWorkspace 'D:\\Departmental Data\\Subjects Data\\July-Dec 2025\\MLOPs\\CI-CD\\Demo1_Jenkins_as_Container'
        }
      }
```

gets replaced by

```
agent any   or   agent { node  { label 'local' } }  or  agent  { label 'local' }
```

A Sample Jenkins file

Notes:

- file:/// → starts the path (3 slashes!). Drive letter (D:) comes directly after the third slash.
- Spaces must be URL encoded as %20. Use forward slashes / even on Windows.
- We set the pipeline definition as "Pipeline Script from SCM"
- agent any is not safe in the previous case unless: we are 100% sure all Jenkins agents (including remote ones or Docker-based agents) have access to the D:\ drive on our local system.
- Risk: Jenkins might pick a remote agent or a container that doesn't have access to D:\..., and the git step will fail with a "repository not found" error.
- The other two options are safe as they ensure the pipeline runs only on an agent labeled 'local', which presumably has the D:\ path available.

A Sample Jenkins file

If the project is available on GitHub, then we need to add the following stage under stages as the first stage:

```
stage('Checkout Code') {
   steps {
      git branch: 'main', url: 'https://github.com/your-username/iris_project.git'
   }
}
```

The following

```
agent {
         node
         {
            label 'local'
            customWorkspace 'D:\\Departmental Data\\Subjects Data\\July-Dec 2025\\MLOPs\\CI-CD\\Demo1_Jenkins_as_Container'
         }
       }
```

gets replaced by

agent any (or same as earlier)

- We set the pipeline definition as "Pipeline Script from SCM"

More on Jenkins

By default, pytest looks for test files like:test_*.py or *_test.py

These files must be present in the working directory (or in a directory specified with -k, -p, or --pyargs).

More on Jenkins

- To Install Required Plugins in Jenkins: Go to Manage Jenkins → Plugins → Available Plugins, and install: Git plugin (to pull code from repo), Pipeline plugin (to define CI/CD steps with Jenkinsfile), **Python plugin (optional, for Python environment management) (may not be required).**

- To configure Python in Jenkins. Go to Manage Jenkins → Global Tool Configuration. Add your Python installation path (example: C:\Python313\python.exe). **(may not be required)**.

- To create a Jenkins Pipeline Job. Click New Item → Enter name (e.g., PythonWebApp-CICD) → Select Pipeline. In the job config: Pipeline Definition → "Pipeline script from SCM" if you're using Git. Or "Pipeline script" if you want to paste Jenkinsfile manually.

- To run the Pipeline. Push your code + Jenkinsfile to repo. In Jenkins UI → click Build Now. We can watch stages: Checkout → Install → Test → Deploy.

- Demo Idea: Make a change in templates/index.html (like "Hello Students"). Commit + push. Trigger Jenkins build. We see: Code pulled, Dependencies checked, Tests run, App redeployed. Open http://localhost:8000 → updated page visible.

- Notes:
  - Stage must have a name (string). However, the name can be anything. The order in which stages are specified matters; they will be executed in that order.

  - To start and stop Jenkins service: net start jenkins, net stop jenkins.

More on Jenkins

Different stages on different nodes:

```
pipeline {
    agent none  // We don't want a default agent here

    stages {
        stage('Build on Node 1') {
            agent { label 'node-1' }  // Specify the label for the first node
            steps {
                echo 'Building on Node 1'
            }
        }

        stage('Build on Node 2') {
            agent { label 'node-2' }  // Specify the label for the second node
            steps {
                echo 'Building on Node 2'
            }
        }
    }
}
```

In this example, node-1 and node-2 would be distinct nodes (which could be Docker containers, VMs, or separate processes), and Jenkins will run different stages on each of these nodes.

Steps in Jenkins UI

1. Log in to Jenkins (http://localhost:8080)
2. New Item: Pipeline
3. General Options (we may not do anything in case of the project as a local directory)
4. Triggers (we may not do anything in case of the project as a local directory)
5. Pipeline
6. Advanced (we may not do anything in case of the project as a local directory)

Note: Jenkins can run the pipeline on the changes if the project is a GitHub repo. It can poll SCM periodically for a local git repo. For a local folder, it can not run the pipeline automatically.

Trigger Type	Requires SCM Changes?	Trigger Delay	Typical Use Case
Manual	No	Immediate	On-demand runs, debugging
Poll SCM	Yes	Poll interval delay	CI on local/remote repos without webhooks
Webhook	Yes	Near instant	CI on remote GitHub/GitLab repos
Scheduled Build	No	Fixed schedule	Nightly builds, periodic tests

Steps in Jenkins UI – General Options

General Option	What It Does	When to Use / Notes
Discard old builds	Automatically deletes old builds and artifacts based on criteria you set	Save disk space by limiting build history (number or age)
Do not allow concurrent builds	Prevents multiple builds of this job running at the same time	Use if builds conflict or share resources and should not overlap
Do not allow the pipeline to resume if the controller restarts	Stops pipeline resuming from where it left off after Jenkins controller restart	Use if pipeline state might get corrupted or you want fresh start after restarts
GitHub project	Add URL of your GitHub repo to the job for easy navigation	It adds a clickable link on your Jenkins job page that points to your GitHub repository. This is mainly for convenience and documentation — it helps users quickly jump from Jenkins to your GitHub repo. It does NOT affect how Jenkins pulls or builds our code. It's purely informational and optional.
Pipeline speed/durability override	Adjust how pipeline execution state is saved (trade-off between speed and fault tolerance)	Use for tuning performance or reliability; advanced use
Preserve stashes from completed builds	Keeps stash data even after builds complete	Useful if you need stash data for later stages or troubleshooting
This project is parameterized	Allows you to add build parameters like strings, booleans, choices	Make your pipeline flexible by accepting input values
Throttle builds	Limit how many concurrent builds of this job can run across nodes	Control resource usage and prevent overload in shared environments

Steps in Jenkins UI – Trigger Options

Trigger Option	What It Does	When to Use	Difference / Notes
Build periodically	Runs the job on a fixed schedule regardless of code changes	For nightly builds, maintenance, or scheduled tasks	Always triggers build at scheduled time, even if no code changes
Poll SCM	Periodically checks the repo for changes and triggers build only if changes found	When webhooks unavailable, want build on code changes only	Only triggers if SCM changes detected, saving resources
GitHub hook trigger for GITScm polling	Trigger build immediately on GitHub push	Most efficient for GitHub-hosted repos	Push-based trigger; no polling needed
Build after other projects	Trigger after specified Jenkins jobs complete	For multi-job pipelines	Chains jobs based on completion
Trigger builds remotely	Trigger via HTTP request with a token	Integrate with external tools or scripts	External system triggers build
GitLab webhook	GitLab push-triggered builds	For GitLab repos	Similar to GitHub webhook

Steps in Jenkins UI – More on Pipeline Option

When you write pipeline scripts (especially Declarative Pipeline or Scripted Pipeline) in Jenkins, the underlying language is Groovy.

The Groovy Sandbox is a security feature that restricts what your Groovy scripts can do, preventing them from running unsafe or potentially harmful code on the Jenkins server.

Why does it matter?

- Without sandbox: Your Groovy script can run any code, including commands that might harm the Jenkins environment or access sensitive data.

- With sandbox enabled: Jenkins limits the script's capabilities to a safe subset of Groovy methods and operations. It blocks potentially dangerous actions.

Steps in Jenkins UI – Schedule Examples

Format: MINUTE HOUR DOM MONTH DOW

Field	Description	Allowed values
MINUTE	Minute (0–59)	0-59, *, */n, H
HOUR	Hour (0–23)	0-23, *, */n, H
DOM	Day of month (1–31)	1-31, *, */n, H
MONTH	Month (1–12)	1-12, *, */n, H
DOW	Day of week (0–7)	0-7 (0 and 7 = Sunday), *, H

Schedule	Meaning
H/5 * * * *	Every 5 minutes, staggered
H H * * *	Once a day at a random time per job
H 2 * * *	Every day at around 2:00 AM
0 9 * * 1-5	At 9 AM, Monday through Friday
*/2 * * * *	Every 2 minutes (not staggered)
15 14 1 * *	At 2:15 PM on the 1st of every month
H/15 9-17 * * 1-5	Every 15 mins during working hours on weekdays

Common symbols	
Symbol	**Meaning**
*	Every possible value (e.g., every minute)
/n	Every n units (e.g., */5 = every 5 mins)
H	"Hash" — Jenkins spreads the load (more below)
H/5	Every 5 units, but staggered per job
,	List (e.g., 0,15,30,45)
-	Range (e.g., 9-17) = 9 AM to 5 PM

Disclaimer

- Content of this presentation is not original and it has been prepared from various sources for teaching purpose.